

B.Tech CSE 2025–29 | Semester II
React JS — Case Study Project

Genetic Algorithm Creature Simulator

An In-Browser Evolutionary Optimisation Engine with Dual Simulation Modes

Submitted By

Ashutosh Pawar

Roll No: 150096725130

ITM Skills University, Kharghar, Navi Mumbai
School of Engineering & Technology
June 2026

1. Abstract

This report documents the design, architecture, and implementation of a Genetic Algorithm Creature Simulator — a fully client-side React JS web application that models evolutionary computation principles inside the browser. The application implements two distinct simulation modes: a Genetic Algorithm Dot Maze (V1), where a population of agents evolves over successive generations to navigate a spatial environment toward a target, and a Greedy Frontier Path Learner (V1.1), a deterministic algorithm that uses ray-casting to progressively construct an obstacle-avoiding path.

The project was built entirely without a backend or external compute server. All evolutionary logic — including roulette-wheel selection, single-point crossover, bit-flip mutation, and Euclidean fitness evaluation — executes in-memory via client-side JavaScript. The application makes extensive use of React's core ecosystem: Context API with useReducer for global state, custom hooks for physics and evolution logic, React Router for multi-page navigation, and localStorage for session persistence.

2. Introduction

Genetic Algorithms (GAs) are a class of evolutionary optimisation techniques inspired by the principles of natural selection and biological evolution. Originally formalised by John Holland in the 1970s, GAs operate on a population of candidate solutions, iteratively improving them through selection, recombination, and mutation over successive generations.

This project implements a GA entirely within a React JS frontend application, demonstrating that meaningful machine learning behaviour can be achieved without any server-side computation. The simulation visualises the emergent intelligence of a population — not through explicit programming of movement rules, but through the natural filtering of genetic material across generations.

2.1 Project Objectives

- Implement a fully functional Genetic Algorithm in client-side JavaScript
- Build a real-time 60fps canvas simulation using `requestAnimationFrame`
- Demonstrate all core GA operators: selection, crossover, mutation, and fitness evaluation
- Apply React best practices: Context API, custom hooks, `useReducer`, React Router
- Persist simulation state across browser sessions using `localStorage`
- Provide a second distinct algorithm (Greedy Frontier Path Learner) for comparative analysis

2.2 Technology Stack

Technology	Version	Purpose
React JS	18.x	UI framework and component architecture
Vite	5.x	Build tool and development server
React Router DOM	6.x	Client-side multi-page routing
Recharts	2.x	Real-time performance chart visualisation
HTML5 Canvas API	Native	60fps creature and obstacle rendering
localStorage API	Native	Session persistence without a database

3. Genetic Algorithm — Theory & Implementation

A Genetic Algorithm mimics biological evolution. A population of candidate solutions (creatures) is evaluated, the fittest are selected as parents, their genetic material is combined and mutated to produce offspring, and this process repeats across generations. Over time, the population converges toward increasingly optimal solutions.

3.1 DNA Representation

Each creature's DNA is an array of 300 movement vectors, where each vector contains an x and y component in the range $[-1, 1]$. On each animation frame, the creature reads the next gene from its DNA array and applies it as an acceleration force.

```
// lib/genome.js - DNA structure
export function createDNA(length) {
  return Array.from({ length }, () => ({
    x: Math.random() * 2 - 1, // -1 to 1
    y: Math.random() * 2 - 1
  }));
}
```

3.2 Fitness Evaluation

Fitness is calculated using the Euclidean distance between a creature's final position and the target. The inverse relationship ensures creatures closer to the target receive higher scores. Bonus and penalty multipliers incentivise target-reaching and penalise obstacle collisions.

```
// lib/fitness.js
export function calcFitness(creature, target) {
  const d = Math.sqrt(
    Math.pow(creature.x - target.x, 2) +
    Math.pow(creature.y - target.y, 2)
  );
  let fitness = 1 / (d + 0.0001); // inverse distance
  if (creature.reached) fitness *= 10; // bonus for reaching
  if (creature.dead) fitness *= 0.1; // penalty for collision
  return fitness;
}
```

3.3 Roulette-Wheel Selection

Parent selection uses a roulette-wheel (fitness-proportionate) strategy. Each creature occupies a "slice" of the wheel proportional to its fitness score. A random spin is performed and whichever creature's slice is landed on becomes a parent. This ensures high-fitness creatures breed more often while preserving population diversity.

```
// lib/genome.js
export function selectParent(population) {
  const total = population.reduce((s, c) => s + c.fitness, 0);
  let rand = Math.random() * total;
  for (const creature of population) {
    rand -= creature.fitness;
    if (rand <= 0) return creature;
  }
  return population[population.length - 1];
}
```

3.4 Single-Point Crossover

Two parent DNA arrays are combined by selecting a random split index. The child inherits the first segment from Parent A and the second segment from Parent B. This models biological recombination and produces offspring that share traits from both parents.

```
// lib/genome.js
export function crossover(dnaA, dnaB) {
  const split = Math.floor(Math.random() * dnaA.length);
  return [...dnaA.slice(0, split), ...dnaB.slice(split)];
}
```

3.5 Bit-Flip Mutation

After crossover, each gene in the child DNA is tested against the mutation rate. If triggered, the gene is replaced with a new random vector. This introduces fresh variation into the gene pool and prevents the population from converging prematurely into local minima.

```
// lib/genome.js
export function mutate(dna, rate) {
  return dna.map(gene =>
    Math.random() < rate
      ? { x: Math.random() * 2 - 1, y: Math.random() * 2 - 1 }
      : gene
  );
}
```

3.6 Elitism

The top 10 creatures by fitness score are carried forward into the next generation unchanged. This elitist strategy guarantees that the best solutions found so far are never lost to crossover or mutation, accelerating convergence while the rest of the population continues to explore.

4. Physics Engine & Canvas Rendering

The simulation runs at 60fps using `requestAnimationFrame`. On every frame, each creature reads its current gene as an acceleration force, which is added to a velocity vector subject to drag. This produces smooth, momentum-based movement rather than jerky teleportation.

```
// hooks/useCanvas.js - physics step
const ACCEL = SPEED * 0.4;
const DRAG = 0.92;

c.vx = (c.vx ?? 0) * DRAG + gene.x * ACCEL;
c.vy = (c.vy ?? 0) * DRAG + gene.y * ACCEL;
c.x += c.vx;
c.y += c.vy;
```

After each position update, three collision checks are performed:

- Boundary check — if the creature exits the canvas bounds, it is marked dead
- Obstacle check — if the creature's position overlaps any obstacle rectangle, it is marked dead
- Target check — if the creature is within `TARGET_RADIUS` pixels of the target, it is marked reached

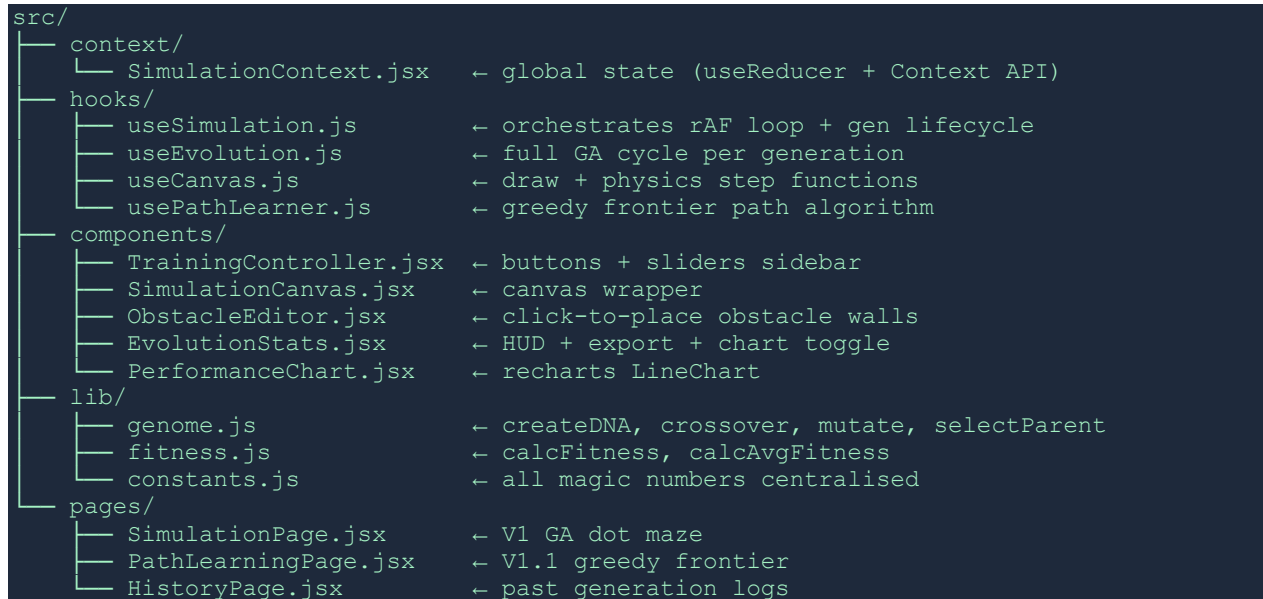
4.1 Canvas Rendering

Creatures are rendered as filled circles. The leading creature (closest to the target among the living) is highlighted in gold (`#fbbf24`) and drawn 2px larger. Reached creatures render green, dead creatures render slate grey, and the rest render indigo.

Visual State	Colour	Meaning
Gold, larger radius	#fbbf24	Current generation leader
Green	#22c55e	Successfully reached the target
Slate grey	#475569	Dead — hit boundary or obstacle
Indigo	#818cf8	Alive and actively moving
Red circle	#ef4444	Target destination
Dark slate	#334155	Obstacle walls

5. React Architecture

5.1 Project Structure



5.2 React Hooks Used

Hook	Where Used	Purpose
useState	TrainingController, EvolutionStats	Slider values, chart toggle visibility
useEffect	useSimulation, useEvolution	Start/stop rAF loop, localStorage sync
useRef	useSimulation, useCanvas	Canvas DOM ref, rAF ID, population mirror
useContext	All components	Access SimulationContext without prop drilling
useReducer	SimulationContext	Complex state transitions via dispatch actions
useCallback	useEvolution, useCanvas	Memoised GA and draw functions
Custom hooks	useSimulation, useEvolution, useCanvas, usePathLearner	Encapsulate complex stateful logic

5.3 Context API & useReducer

All global simulation state — population array, generation count, running/paused flags, obstacle positions, target coordinates, slider parameters, and stats history — lives in `SimulationContext`. The context uses `useReducer` internally, exposing a dispatch function to all child components. This eliminates prop drilling entirely across the component tree.

Dispatch Action	Effect
SET_RUNNING	Toggle <code>isRunning</code> flag, start/stop the rAF loop
SET_PAUSED	Toggle <code>isPaused</code> flag without resetting frame counter
NEXT_GENERATION	Replace population array, increment generation, update stats
SET_PARAM	Update any slider parameter (<code>popSize</code> , <code>mutationRate</code> , <code>genDuration</code>)
ADD_OBSTACLE	Append a new obstacle rectangle to the obstacles array
CLEAR_OBSTACLES	Reset obstacles to empty array
RESET	Regenerate fresh population, zero generation counter
LOAD_FROM_STORAGE	Restore generation count and stats history from <code>localStorage</code>

5.4 React Router

The application uses `React Router v6` with three routes, all wrapped inside the `SimulationProvider` so context state is shared across pages:

```
// App.jsx
<BrowserRouter>
  <SimulationProvider>
    <Routes>
      <Route path="/" element={<SimulationPage />} />
      <Route path="/path" element={<PathLearningPage />} />
      <Route path="/history" element={<HistoryPage />} />
    </Routes>
  </SimulationProvider>
</BrowserRouter>
```


6. UI Components

6.1 TrainingController

The sidebar control panel implementing the PS-specified `AlgorithmActionStrip` and `HyperparameterForm`. Contains all four required action buttons and all three parameter sliders.

Control	Type	Function
Start Evolutionary Cycles	Button	Dispatches <code>SET_RUNNING</code> → begins rAF loop
Pause Training Operations	Button	Dispatches <code>SET_PAUSED</code> → halts loop mid-generation
Skip Active Generation Timer	Button	Calls <code>endGeneration()</code> immediately
Purge Gene Pools	Button	Dispatches <code>RESET</code> → fresh random population
Population Capacity	Slider (10–500)	Rebuilds population at new size via <code>SET_PARAM</code>
Mutation Rate	Slider (0–100%)	Updates <code>mutationRate</code> (stored as 0–1 decimal)
Generational Lifespan Timer	Slider (1–30s)	Time limit per generation before auto-advance

6.2 EvolutionStats (Telemetry HUD)

A bottom-docked status panel implementing the PS-specified `EvolutionTelemetryHUD`. Displays five live metrics calculated in-memory each generation.

Metric	Calculation
Maximum Fitness	<code>Math.max(...population.map(c => c.fitness))</code>
Average Generation Fitness	<code>sum(fitness) / population.length</code>
Total Elapsed Generations	generation counter from context state
Matrix Synthesis Time (ms)	<code>performance.now()</code> diff around <code>runGeneration()</code>
Reached Target	count of creatures with <code>reached === true</code>

Additionally provides two utility actions:

- Export Elite DNA to JSON — serialises the highest-fitness creature's DNA array and triggers a browser file download
- Toggle Real-Time Performance Chart — shows/hides a Recharts LineChart of max and average fitness across all generations

6.3 ObstacleEditor

Allows users to click anywhere on the simulation canvas to place rectangular wall obstacles. Obstacle coordinates are stored in the SimulationContext obstacles array and read by both the physics engine and the canvas renderer. A Clear Obstacles button dispatches CLEAR_OBSTACLES to reset the environment.

6.4 PerformanceChart

A Recharts LineChart component that visualises two data series across all completed generations: maximum fitness (indigo line) and average fitness (green line). The x-axis represents generation number and the y-axis represents fitness score. This chart allows the researcher to observe evolutionary convergence visually over time.

7. Greedy Frontier Path Learner (V1.1)

As an extension beyond the core GA requirement, a second algorithm was implemented: a greedy frontier path learner. This provides a meaningful point of comparison between stochastic evolutionary search (the GA) and deterministic heuristic search.

7.1 Algorithm Description

The path learner works by casting a fan of rays outward from the current frontier point. Each ray is tested for obstacle intersection. The ray endpoint that lands closest to the target (while avoiding obstacles) becomes the next frontier point. This process repeats until the target is within reach, building a chain of waypoints that forms a navigable path.

Phase	Description
Searching	Rays fan out from the frontier; the best endpoint becomes the next waypoint
Done	A clear line of sight to the target has been established
Animating	A smooth agent traverses the built waypoint chain start to finish

7.2 GA vs Path Learner — Comparison

Dimension	Genetic Algorithm	Path Learner
Type	Stochastic / evolutionary	Deterministic / heuristic
Population	200 simultaneous agents	Single frontier point
Learning	Improves across generations	Builds path in one pass
Obstacle handling	Agents die on collision	Rays avoid obstacles explicitly
Output	Evolved movement DNA	Explicit waypoint chain
Best for	Complex fitness landscapes	Clear navigable environments

8. Data Persistence — localStorage

The application persists simulation data across browser sessions without any server or database. Three items are saved to localStorage at the end of every generation:

Key	Value Stored	Used For
eliteDNA	JSON array of top creature's 300-gene DNA	Reference across sessions
generation	Integer generation counter	Restore progress on refresh
statsHistory	JSON array of {gen, max, avg} per generation	Reload performance chart history

On application mount, a useEffect hook reads these values and dispatches `LOAD_FROM_STORAGE` to restore the previous session's context state. This allows a researcher to close the browser and resume analysis without losing historical data.

9. Development Bug Log

Six bugs were encountered and resolved during development. Each was documented with root cause analysis and fix in a BUGSHEET.md file committed to the project repository.

Bug ID	File	Type	Severity	Summary
BUG-001	useCanvas.js	Duplicate import	Build-breaking	SPEED imported twice as alias — ESLint error
BUG-002	useSimulation.js	Ref in render	Build-breaking	Ref.current assigned during render — hooks linter violation
BUG-003	SimulationContext.jsx	ESLint config	Build-breaking	Mixed component/utility exports broke Vite fast-refresh HMR
BUG-004	useSimulation.js	Stale closure	Runtime silent	Rapid skip clicks read stale population — fixed with sync ref update
BUG-005	usePathLearner.js	Wrong HUD value	UI cosmetic	Distance HUD showed non-zero after target reached — fixed setBestDist(0)
BUG-006	usePathLearner.js	Stale canvas draw	UI cosmetic	Ray fan lingered after search completed — fixed with clearRays + redraw

10. Future Scope — V2 Tower Climb

The architecture of this project is explicitly designed to extend into a gamified V2 mode: a Tower Climb in which three species (Giraffe, Ape, Mystical) evolve floor by floor up a tower to defeat a mega boss — King Kong — at the summit.

10.1 V2 Core Changes

Component	V1 (Current)	V2 (Planned)
DNA shape	Movement vectors [{x, y}]	Trait values [neck, str, fire, speed, armor]
Fitness function	Euclidean distance to target	Damage dealt vs floor enemy's stats
Environment	2D canvas maze	Tower floors with escalating difficulty
Visualisation	Coloured dots	AI-generated creature images per floor
Species	Homogeneous population	3 distinct species with base trait profiles

10.2 Trait Cap System

Each floor defines a maxTotal (sum of all traits) and maxSingle (per-trait ceiling) that increase as players climb. This prevents creatures from maxing all traits early and forces strategic specialisation across floors. The cap is enforced automatically by reading FLOORS[floorIndex] after every crossover operation — no hardcoded values, no manual switches.

Floor	Enemy	Max Total Traits	Max Single Trait	Key Requirement
1	Goblin	10	3	str ≥ 2
2	Orc	15	5	str ≥ 3, fire ≥ 1
3	Dragon	20	7	fire ≥ 4, armor ≥ 2
4 (Boss)	King Kong	30	10	neck ≥ 5, str ≥ 5, fire ≥ 6

11. Conclusion

This project demonstrates that a fully functional evolutionary computation system can be built entirely in the browser using React JS, without any backend infrastructure, AI APIs, or external compute. The Genetic Algorithm engine — encompassing roulette-wheel selection, single-point crossover, bit-flip mutation, elitism, and Euclidean fitness evaluation — operates at 60fps in real time, producing visibly emergent intelligent behaviour across generations.

The project goes beyond the minimum PS requirements by implementing a second distinct algorithm (Greedy Frontier Path Learner) with its own dedicated page, providing a meaningful contrast between stochastic evolutionary search and deterministic heuristic pathfinding. The codebase is architected for clean extensibility, with a V2 Tower Climb mode fully designed and ready to implement.

From an academic standpoint, the project demonstrates practical command of React's most important patterns: Context API with useReducer for global state, custom hooks for physics and evolution logic, requestAnimationFrame for performant animation, and React Router for multi-page navigation — all applied to a non-trivial, genuinely interesting problem.

12. References

#	Reference
1	Holland, J.H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.
2	Mitchell, M. (1998). An Introduction to Genetic Algorithms. MIT Press.
3	React Documentation — Hooks API Reference. https://react.dev/reference/react
4	MDN Web Docs — requestAnimationFrame. https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame
5	Recharts Documentation. https://recharts.org/en-US/api
6	React Router v6 Documentation. https://reactrouter.com/en/main
7	Vite Documentation. https://vitejs.dev/guide

— End of Report —